COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: Tanuj Tekkale | UT EID: tt27868

Repository: `https://github.com/TJTheCoder/COE-397L-Midterm`

1. System Overview

The agent takes a `LabRequest` with the user's authenticated identity and any project, equipment, and times that have already been resolved, and I use those fields to check the model's proposed actions, so if the request names Alice and tensile-01 the model cannot just reserve a different machine for Bob. The original request text stays separate from this metadata.

The model returns one of three decisions: call a tool, ask for clarification, or give a final response. Pydantic selects the decision type using its `kind` field. The loop accepts at most ten decisions by default. Before executing a tool, it checks the arguments and request binding. A reservation also needs all ten facts in the supplied `ReservationContext` to be true. These cover the required policy lookup, authorization and resource checks, training, availability, calibration, maintenance, and after-hours approval.

Each tool proposal and its result go into the trace, including denied calls which never reach the tool dispatcher, and the next model turn receives both the proposal and the observation so it can see what went wrong and respond to the failed check. The prompt tells the model to treat retrieved text and tool observations as untrusted data, rather than follow instructions inside them. Before returning a final response, the code checks its citations and completed-action list against what actually happened. A clarification question ends the run without another tool call.

2. Public Benchmark Results

Table 1: Results recorded in `public_benchmark.json`.

Metric	Value
Scenarios	6
Expected-status success rate	100%
Invalid structured response rate	0%
Unsafe action attempt rate	0%
Unnecessary refusal rate	0%
Clarification accuracy	100%
Average tool calls per scenario	2.67

All six public scenarios return the expected status. The tensile tester case succeeds after four calls: policy search, user authorization, resource status, and reservation. The missing laser training, expired robot calibration, missing after-hours approval, and scheduling conflict cases are blocked. Each of those runs makes three read-only calls and no reservation attempt. The ambiguous request asks for exact times without calling a tool. Across the benchmark, there are 16 tool proposals: 15 read-only calls and one successful reservation.

There were no failed or unexpected scenarios, although the zero unsafe-attempt rate needs some explanation because the supplied scripts stop on their own in the blocked cases and never actually ask the gate to reject a reservation. I added tests which propose unsafe actions directly for that reason. The benchmark shows that these six scripted runs behave as expected, but it does not tell me how well a live model would understand a new request.

3. Student Tests

All 47 tests pass, including the 23 supplied tests and 24 added cases in `tests/test_student_cases.py`. I used `ScriptedLLM` for the added cases so the decisions stay the same on each run.

The first group changes one request field at a time, either the user, project, resource, start, or end, and after the normal prerequisite checks the model proposes the altered reservation so I can check that the call is denied before dispatch and the simulator state has not changed. Another five cases propose a reservation despite missing training, expired calibration, maintenance, missing after-hours approval, or an occupied time slot. These cases check that the application enforces policy even when the model proposes the wrong action.

For the contradictory-response test, I made the reservation tool return a conflict and then had the model claim success anyway, which should make the agent reject the final response with `invalid_final_response` since no reservation was actually created. Other tests cover invented citations and a successful reservation followed by a blocked response or a clarification question. There is also a duplicate-reservation test: even with a fresh status lookup, the second reservation is denied and only one new reservation exists.

The remaining cases cover malformed decisions, a missing argument, an unresolved request field, and zero-step and two-step limits. In the injection test, a retrieved passage says to skip authorization and reserve immediately. The scripted model then proposes exactly that. The gate still rejects the call because the required authorization and resource checks are missing. This checks the application's response to one injected action, rather than relying on the prompt alone.

4. Implementation Decisions

I left the supplied infrastructure unchanged. The decision schemas reject extra fields, and the loop validates tool arguments before passing them to the policy-context helper. This avoids trying to build context from incomplete arguments. An invalid call becomes a failure observation that the model can see. If the context calculation raises an input-related error, the call is denied.

I also reject reservations when a required field in the trusted request is unresolved, because letting the model fill it in would allow a guess to become an authorized action and I wanted the caller to resolve that information before starting another run. Each failed policy check has a separate error identifier, which makes the trace easier to inspect. Once a mutation succeeds, the loop blocks further mutations for that request. This prevents duplicates before they reach the simulator.

The final-response validator checks citations against successful searches and matches action claims to successful mutating calls. Repeating a claim does not create another successful action. A blocked response or clarification cannot hide a reservation that already succeeded. Separate termination reasons identify malformed decisions, inconsistent final responses, clarification, normal completion, and the step limit. If a later decision fails after a reservation succeeds, the trace keeps that success; returning a failure does not undo the reservation.

There are still gaps in these checks, for example a citation can have been retrieved without really supporting the response, and the validator does not check every sentence of the message or independently verify the policy facts and time calculations from the supplied helpers. The tests cover specific inputs, not every possible sequence of decisions, and they do not test concurrent access or a live model. Passing them is useful evidence, but it is not a proof that every run will be correct. The separate `data_flow_diagram.pdf` shows the successful tensile tester scenario.